

Anyanwu Somtochukwu

World Airline Route Search

CSCE 2110
Project report

Design Overview

The goal of this project is to build a program that reads a file of airline flight routes and answers four different search questions about those routes. The flight data is stored as a directed graph, where each city is a node and each flight route is a directed edge. The four questions ask about shortest paths, multi-stop routes, full city tours, and optimal meeting points.

The program is written in C++ and takes command line arguments to determine which question to answer and which cities to use as input. The program reads from a file called flight.txt which lists each city and the cities it flies to directly.

Data Structures

Adjacency List

The graph is stored using an adjacency list implemented with an unordered map where each key is a city name (string) and each value is a vector of destination city names (vector of strings). This structure was chosen because the flight graph is sparse, meaning most cities only fly to a small number of other cities rather than every city in the network. An adjacency list only stores the edges that exist, making it far more memory efficient than a matrix in this case.

A 2D matrix was considered but rejected because it requires integer indices rather than string city names, and it allocates memory for all possible city pairs even when most of those pairs have no direct flight. For a network with 100 cities, a matrix would allocate 10,000 cells when only a fraction of those represent real routes.

An edge list was also considered but rejected because finding all neighbors of a given city requires scanning the entire list, which is slow during BFS traversal. The adjacency list allows constant time neighbor lookup by city name.

The unordered map specifically was chosen over a regular map because it uses a hash table internally, giving $O(1)$ average case lookup time rather than $O(\log n)$ for a sorted map. Since BFS performs many city lookups during traversal, this difference matters for performance.

Algorithms

Breadth-First Search (BFS)

BFS is the core algorithm used for questions 1, 2, and 4. It was chosen because the graph is unweighted, meaning every flight connection costs exactly 1. In an unweighted graph, BFS guarantees the shortest path by exploring all cities at distance 1 before exploring cities at distance 2, and so on. The first time BFS reaches any city, that path is guaranteed to be the shortest.

Alternative algorithms were considered. Dijkstra's algorithm works on weighted graphs and uses a priority queue to always process the closest unvisited node next. It would work here but adds unnecessary complexity since all edges have equal weight. Bellman-Ford handles negative weights and detects negative cycles, but is slower at $O(V \times E)$ and is unnecessary for this problem. DFS was rejected for shortest path queries because it does not guarantee the shortest path.

The BFS implementation returns both a distance map and a parent map. The distance map stores the shortest distance from the source to every reachable city. The parent map stores which city each city was reached from, which allows the actual path to be reconstructed by tracing backwards from the destination.

Depth-First Search with BFS Return (Question 3)

Question 3 asks the program to visit all reachable cities and return to the starting city using as few connections as possible. This is a variation of the Travelling Salesman Problem (TSP), which is NP-hard in general. A pure brute force approach that tries every possible ordering would have $O(n!)$ time complexity, which is completely impractical for any graph with more than about 15 cities.

The approach used is a hybrid of DFS and BFS. DFS is used to traverse as many cities as possible by following real directed flight edges in the graph. DFS naturally goes deep along available paths before backtracking, which allows it to discover long chains of connected cities. Once DFS has visited all reachable cities, BFS is used to find the shortest path from the last visited city back to the starting city.

The hybrid DFS and BFS approach for question 3 was inspired by discussions during the in-class presentations, where comparing different approaches highlighted the limitations of the greedy nearest neighbor method and suggested that DFS traversal combined with BFS for the return path could produce better results.

This approach does not always find the globally optimal tour, but it runs efficiently and always follows real flight connections. The directed nature of the graph means a complete round trip is not always possible, in which case the program reports how far the tour reached before being unable to return.

Time Complexity

Question	Algorithm	Time Complexity
----------	-----------	-----------------

Q1: Route with connection limit	BFS	$O(V + E)$
Q2: Route through waypoints	BFS x 4	$O(V + E)$
Q3: Visit all cities and return	DFS + BFS	$O(V + E)$
Q4: Find meeting city	BFS x 3	$O(V + E)$

V represents the number of cities in the graph and E represents the number of directed flight routes. BFS visits each city and each edge at most once, making it linear in the size of the graph. For questions 2 and 4, multiple BFS calls are made but each is still $O(V + E)$, so the overall complexity remains linear.

Scalability

The BFS based approach scales well for larger graphs. Since BFS is $O(V + E)$, it handles hundreds or thousands of cities efficiently as long as the graph fits in memory. The adjacency list representation also scales well since memory usage grows proportionally to the number of edges rather than the square of the number of cities.

Question 3 is the main scalability concern. The DFS plus BFS hybrid approach runs in $O(V + E)$ but does not guarantee the optimal tour order. For very large graphs, the tour produced may be significantly longer than the true optimum. A more accurate approach such as dynamic programming TSP would give better results but runs in $O(2^n \times n)$ time, which is impractical for large n .

If the flight network were to scale to thousands of cities globally, the BFS based questions (Q1, Q2, Q4) would still perform well. Question 3 would need a more sophisticated heuristic or approximation algorithm to produce good results at scale. One possible optimization would be to precompute all pairwise shortest distances using BFS from every node and store the results, then apply a nearest neighbor heuristic or 2-opt improvement to the tour.

Optimization Possibilities

Several optimizations could further improve the implementation. Bidirectional BFS runs BFS simultaneously from both the source and destination, meeting in the middle. This can reduce the search space significantly for long paths. For question 4, all three BFS searches could be run in a more memory efficient way by sharing distance computations where possible.

For question 3, a nearest neighbor greedy heuristic would be faster than DFS for finding a reasonable tour order, though it still would not guarantee the optimal result. Adding 2-opt or 3-opt local search improvements on top of an initial greedy tour could produce much better results without the exponential cost of brute force.

The current parser reads the flight.txt file line by line and strips formatting characters. This could be optimized to read the file in larger chunks, though for the file sizes used in this project the current approach is fast enough.

Remarks

The most challenging aspect of this project was handling the directed nature of the flight graph. Unlike undirected graphs where any edge can be traversed in both directions, directed edges mean that a flight from city A to city B does not guarantee a flight from B back to A. This creates situations where question 3 cannot complete a full round trip, which the program handles by reporting a partial tour and noting that no return path exists.

The file parser also required careful handling because the flight.txt format uses multi-line entries with inconsistent spacing around the To : label. The final implementation strips any line beginning with To before extracting the city name, which handles all cases observed in the dataset.

The hybrid DFS and BFS approach for question 3 was inspired by discussions during the in-class presentations, where comparing different approaches highlighted the limitations of the greedy nearest neighbor method and suggested that DFS traversal combined with BFS for the return path could produce better results.